

Deep Reinforcement Learning - Assignment 1

Part 1: Adaptive Treatment Recommendation using Multi-Armed Bandits

Team Number: 165

Team Details:

AMARJIT SINGH 2025aa05130

MANJUNATH S 2025aa05935

SAURAV SINHA 2025aa05427

SOUMYA KANT DAS 2025aa05372

VIPUL PRAKASH CHAUDHARI 2025aa05991

Execution Environment Info

Fetching timestamp, hostname, and platform details to verify virtual lab execution.

```
In [9]: import datetime
import socket
import platform
import os

# Print execution environment details for submission verification
exec_timestamp = datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S')
vm_hostname = socket.gethostname()
os_info = platform.platform()
py_version = platform.python_version()
user_name = os.getenv('USER', os.getenv('USERNAME', 'unknown'))

print('=' * 60)
print('EXECUTION ENVIRONMENT DETAILS')
print('=' * 60)
print(f'Timestamp      : {exec_timestamp}')
print(f'Hostname / VM ID: {vm_hostname}')
print(f'OS Platform      : {os_info}')
print(f'Python Version   : {py_version}')
print(f>Login User       : {user_name}')
print('=' * 60)

=====
EXECUTION ENVIRONMENT DETAILS
=====
Timestamp      : 2026-06-08 09:45:44
Hostname / VM ID: 2025aa05991
OS Platform      : Linux-5.14.0-503.14.1.el9_5.x86_64-with-glibc2.34
Python Version   : 3.12.7
Login User       : cloud
=====
```

Imports

Imports necessary libs and packages

```
In [11]: import numpy as np
import pandas as pd
import random
import math
import time
import matplotlib.pyplot as plt
import matplotlib
import warnings

warnings.filterwarnings('ignore')
matplotlib.rcParams['figure.dpi'] = 110
matplotlib.rcParams['axes.grid'] = True

print('Imports done.')
```

Imports done.

Configuration - Group-Dependent Parameters

Our group number is 165. Hence $G = 165$. The number of arms (K) and the hidden probabilities are derived from G as specified in the assignment.

```
In [13]: # Our team/group number
G = 165

# Seed both RNGs for reproducibility
random.seed(G)
np.random.seed(G)

# Number of medicines (arms) - derived from group number
K = (G % 3) + 5

# Hidden success probabilities for each medicine
hidden_probs = [round(0.4 + ((G + i) % 6) * 0.07, 4) for i in range(K)]

NUM_PATIENTS = 1000 # total iterations / patients

print(f'Group Number (G)      : {G}')
print(f'Number of Medicines (K): {K}')
print(f'Hidden Success Probabilities:')
for idx, prob in enumerate(hidden_probs):
    print(f'  Medicine {idx}: P = {prob}')
print(f'\nBest possible medicine : Medicine {np.argmax(hidden_probs)} (P = {max(hidden_probs)})')
```

```
Group Number (G)      : 165
Number of Medicines (K): 5
Hidden Success Probabilities:
  Medicine 0: P = 0.61
  Medicine 1: P = 0.68
  Medicine 2: P = 0.75
  Medicine 3: P = 0.4
  Medicine 4: P = 0.47
```

```
Best possible medicine : Medicine 2 (P = 0.75)
```

Task 1 - Dataset Design

We generate 1000 patient records. Each patient gets a severity score based on their ID.

The columns `assigned_medicine`, `clinical_outcome`, and `utility_score` will be populated dynamically during each bandit strategy run.

```
In [15]: def build_patient_dataframe(n_patients=1000):
    """
    Create the initial patient dataframe with IDs and severity scores.
    Severity cycles from 1 to 5 based on patient_id mod 5.
    Other columns (assigned_medicine, clinical_outcome, utility_score)
    are placeholders filled during simulation.
    """
    patient_ids = list(range(n_patients))
    severity_vals = [(pid % 5) + 1 for pid in patient_ids]

    df = pd.DataFrame({
        'patient_id': patient_ids,
        'severity_score': severity_vals,
        'assigned_medicine': -1,      # filled during bandit run
        'clinical_outcome': -1,      # filled during bandit run
        'utility_score': 0.0         # filled during bandit run
    })
    return df

def pull_medicine_arm(med_index, patient_severity, rng=None):
    """
    Simulate administering medicine med_index to a patient.
    Returns (clinical_outcome, utility_score).

    clinical_outcome: 1 with probability hidden_probs[med_index], else 0
    utility_score: outcome * (1 - severity/10), so sicker patients get less benefit
    """
    success_prob = hidden_probs[med_index]

    if rng is not None:
```

```

outcome = 1 if rng.random() < success_prob else 0
else:
    outcome = 1 if np.random.random() < success_prob else 0

# utility accounts for severity - sicker patients benefit less even if cured
utility = outcome * (1.0 - patient_severity / 10.0)
return outcome, utility

# Build the base dataset
patient_df = build_patient_dataframe(NUM_PATIENTS)
print(f'Dataset created with {len(patient_df)} patient records.\n')
print('--- First 10 rows ---')
print(patient_df.head(10).to_string(index=False))

```

Dataset created with 1000 patient records.

```

--- First 10 rows ---
patient_id  severity_score  assigned_medicine  clinical_outcome  utility_score
0           1              -1                 -1             0.0
1           2              -1                 -1             0.0
2           3              -1                 -1             0.0
3           4              -1                 -1             0.0
4           5              -1                 -1             0.0
5           1              -1                 -1             0.0
6           2              -1                 -1             0.0
7           3              -1                 -1             0.0
8           4              -1                 -1             0.0
9           5              -1                 -1             0.0

```

```

In [16]: # Show severity distribution across all patients
sev_counts = patient_df['severity_score'].value_counts().sort_index()
print('Severity Distribution:')
for sev, cnt in sev_counts.items():
    print(f'Severity {sev}: {cnt} patients')

fig, ax = plt.subplots(figsize=(7, 4))
ax.bar(sev_counts.index, sev_counts.values, color='steelblue', edgecolor='black', alpha=0.85)
ax.set_xlabel('Severity Score')
ax.set_ylabel('Number of Patients')
ax.set_title('Patient Severity Distribution (1000 Patients)')
ax.set_xticks(sev_counts.index)
plt.tight_layout()
plt.show()

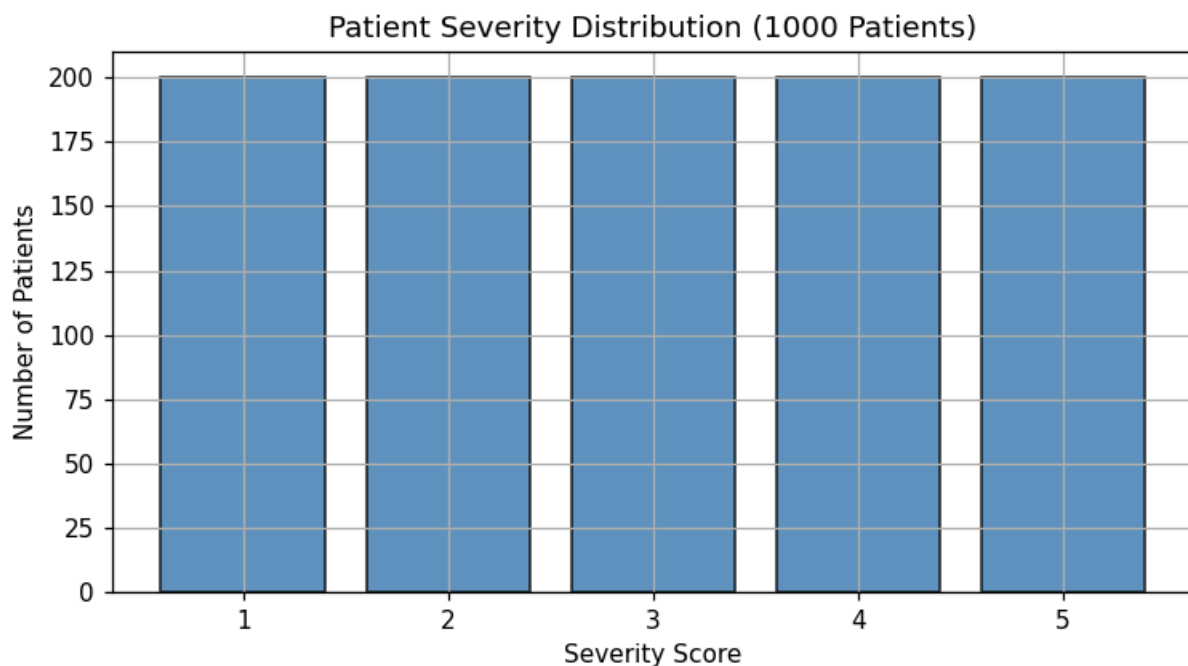
```

Severity Distribution:

```

Severity 1: 200 patients
Severity 2: 200 patients
Severity 3: 200 patients
Severity 4: 200 patients
Severity 5: 200 patients

```



```

In [17]: # Medicine statistics table - show hidden probabilities alongside expected utility
med_stats = pd.DataFrame({
    'Medicine': [f'Med_{i}' for i in range(K)],
    'Hidden_P': hidden_probs,
    'Expected_Utility_Sev1': [round(p * (1 - 1/10), 4) for p in hidden_probs],

```

```

'Expected_Utility_Sev3': [round(p * (1 - 3/10), 4) for p in hidden_probs],
'Expected_Utility_Sev5': [round(p * (1 - 5/10), 4) for p in hidden_probs]
})
print('Medicine Statistics (hidden info - for reference only):')
print(med_stats.to_string(index=False))

```

Medicine Statistics (hidden info - for reference only):

Medicine	Hidden_P	Expected_Utility_Sev1	Expected_Utility_Sev3	Expected_Utility_Sev5
Med_0	0.61	0.549	0.427	0.305
Med_1	0.68	0.612	0.476	0.340
Med_2	0.75	0.675	0.525	0.375
Med_3	0.40	0.360	0.280	0.200
Med_4	0.47	0.423	0.329	0.235

Task 2 - Immediate Exploitation Strategy

Idea: Try each medicine exactly 10 times to get an initial estimate, then forever exploit the medicine with the highest observed mean clinical outcome.

This is a pure greedy strategy - no further exploration after the initial phase.

```

In [19]: def run_greedy_strategy(n_patients, n_medicines, init_pulls=10, seed_val=None):
    """
    Greedy-after-exploration strategy (Immediate Exploitation).

    Phase 1 (exploration): try each medicine exactly init_pulls times via round-robin.
    Phase 2 (exploitation): always pick the medicine with the highest observed mean.
    No further exploration after the initial phase.
    """
    rng = np.random.RandomState(seed_val if seed_val is not None else G)

    arm_total_outcome = np.zeros(n_medicines) # cumulative successes per arm
    arm_pull_count = np.zeros(n_medicines) # number of pulls per arm
    cum_rewards = []
    running_total = 0.0
    records = []

    exploration_budget = init_pulls * n_medicines # total pulls in exploration phase

    for t in range(n_patients):
        severity = (t % 5) + 1

        if t < exploration_budget:
            # Phase 1: round-robin across all medicines
            chosen_arm = t % n_medicines
        else:
            # Phase 2: pure exploitation - pick best mean
            arm_means = arm_total_outcome / np.maximum(arm_pull_count, 1)
            chosen_arm = int(np.argmax(arm_means))

        outcome, utility = pull_medicine_arm(chosen_arm, severity, rng=rng)

        # Update arm statistics
        arm_pull_count[chosen_arm] += 1
        arm_total_outcome[chosen_arm] += outcome
        running_total += utility
        cum_rewards.append(running_total)

        records.append({
            'patient_id': t,
            'severity_score': severity,
            'assigned_medicine': chosen_arm,
            'clinical_outcome': outcome,
            'utility_score': utility
        })

    final_means = arm_total_outcome / np.maximum(arm_pull_count, 1)
    result_df = pd.DataFrame(records)

    return {
        'cum_rewards': cum_rewards,
        'arm_counts': arm_pull_count.copy(),
        'arm_means': final_means.copy(),
        'df': result_df,
        'total_reward': running_total
    }

```

Run the greedy strategy

```

greedy_result = run_greedy_strategy(NUM_PATIENTS, K, init_pulls=10, seed_val=G)

print('=== Immediate Exploitation (Greedy) Results ===')
print(f'Total cumulative reward: {greedy_result["total_reward"]:.4f}\n')
print('Medicine selection counts:')
for i in range(K):
    cnt = int(greedy_result['arm_counts'][i])
    mean_val = greedy_result['arm_means'][i]
    print(f' Medicine {i}: selected {cnt} times, estimated mean = {mean_val:.4f}')
print(f'\nExploration phase used first {10*K} patients ({10} per medicine).')
print(f'After that, all remaining patients received Medicine {int(np.argmax(greedy_result["arm_means"]))}')

```

=== Immediate Exploitation (Greedy) Results ===
Total cumulative reward: 485.6000

Medicine selection counts:

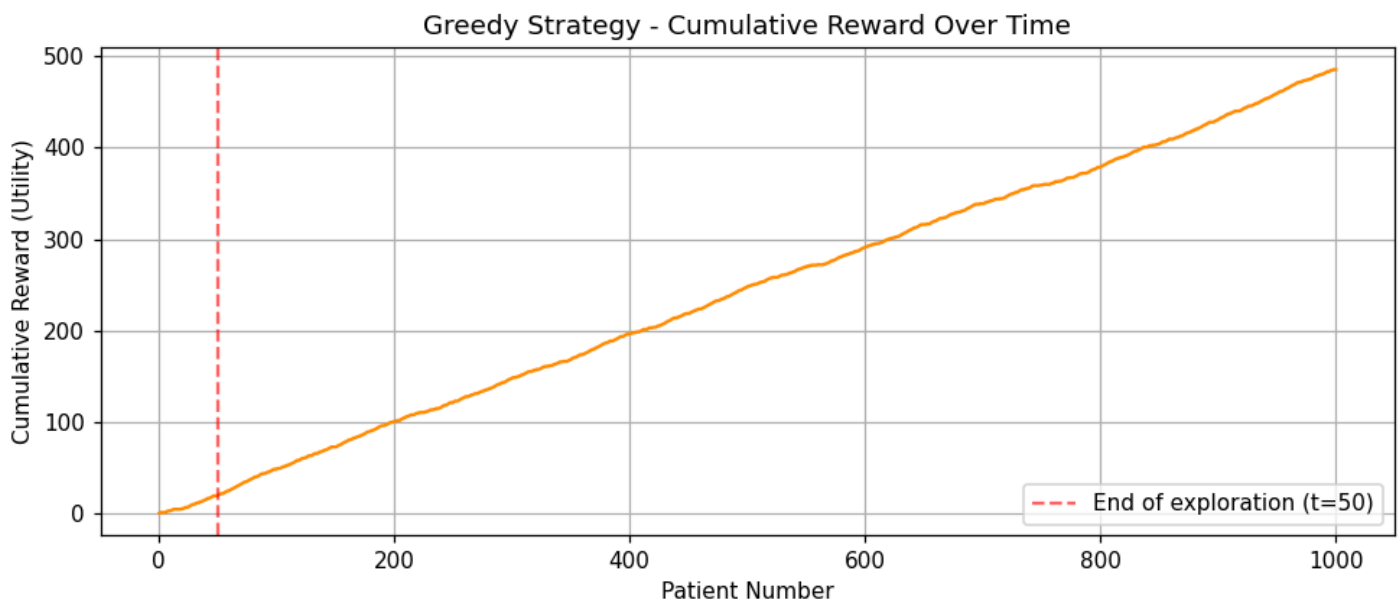
Medicine 0: selected 10 times, estimated mean = 0.4000
Medicine 1: selected 960 times, estimated mean = 0.7000
Medicine 2: selected 10 times, estimated mean = 0.6000
Medicine 3: selected 10 times, estimated mean = 0.3000
Medicine 4: selected 10 times, estimated mean = 0.6000

Exploration phase used first 50 patients (10 per medicine).
After that, all remaining patients received Medicine 1

```

In [20]: # Plot cumulative reward growth for greedy strategy
fig, ax = plt.subplots(figsize=(9, 4))
ax.plot(range(1, NUM_PATIENTS + 1), greedy_result['cum_rewards'], color='darkorange', linewidth=1.5)
ax.axvline(x=10 * K, color='red', linestyle='--', alpha=0.6, label=f'End of exploration (t={10*K})')
ax.set_xlabel('Patient Number')
ax.set_ylabel('Cumulative Reward (Utility)')
ax.set_title('Greedy Strategy - Cumulative Reward Over Time')
ax.legend()
plt.tight_layout()
plt.show()

```



Task 3 - Controlled Clinical Trial (Epsilon-Greedy)

With probability `epsilon` we explore (pick a random medicine), otherwise we exploit the best known medicine. We test three values: **0.10**, **0.01**, and **0.50**.

```

In [22]: def run_epsilon_greedy(n_patients, n_medicines, epsilon, seed_val=None):
    """
    Epsilon-greedy strategy (Controlled Clinical Trial).

    With probability epsilon, pick a random arm (exploration).
    Otherwise, pick the arm with the highest observed mean (exploitation).
    Each arm gets 1 initial pull to avoid cold-start division-by-zero.
    """
    rng = np.random.RandomState(seed_val if seed_val is not None else G)

    arm_total_outcome = np.zeros(n_medicines)
    arm_pull_count = np.zeros(n_medicines)
    cum_rewards = []
    running_total = 0.0

```

```

exploration_steps = 0
records = []

for t in range(n_patients):
    severity = (t % 5) + 1

    # Ensure every arm gets at least 1 pull first
    if t < n_medicines:
        chosen_arm = t
        exploration_steps += 1
    elif rng.random() < epsilon:
        # Explore: random arm selection
        chosen_arm = rng.randint(0, n_medicines)
        exploration_steps += 1
    else:
        # Exploit: pick best known arm
        arm_means = arm_total_outcome / np.maximum(arm_pull_count, 1)
        chosen_arm = int(np.argmax(arm_means))

    outcome, utility = pull_medicine_arm(chosen_arm, severity, rng=rng)

    arm_pull_count[chosen_arm] += 1
    arm_total_outcome[chosen_arm] += outcome
    running_total += utility
    cum_rewards.append(running_total)

    records.append({
        'patient_id': t,
        'severity_score': severity,
        'assigned_medicine': chosen_arm,
        'clinical_outcome': outcome,
        'utility_score': utility
    })

final_means = arm_total_outcome / np.maximum(arm_pull_count, 1)
result_df = pd.DataFrame(records)

return {
    'cum_rewards': cum_rewards,
    'arm_counts': arm_pull_count.copy(),
    'arm_means': final_means.copy(),
    'df': result_df,
    'total_reward': running_total,
    'explore_count': exploration_steps
}

```

Run with three different epsilon values as required

```
eps_configs = [0.10, 0.01, 0.50]
```

```
eps_results = {}
```

```

for eps in eps_configs:
    result = run_epsilon_greedy(NUM_PATIENTS, K, epsilon=eps, seed_val=6)
    eps_results[eps] = result
    print(f'--- Epsilon = {eps} ---')
    print(f'  Total cumulative reward: {result["total_reward"]:.4f}')
    print(f'  Exploration steps: {result["explore_count"]} / {NUM_PATIENTS}')
    print(f'  Arm counts: {[int(x) for x in result["arm_counts"]]}')
    print(f'  Arm means : {[round(x, 4) for x in result["arm_means"]]}')
    print()

```

```
--- Epsilon = 0.1 ---
```

```
  Total cumulative reward: 494.9000
```

```
  Exploration steps: 113 / 1000
```

```
  Arm counts: [15, 265, 662, 23, 35]
```

```
  Arm means : [0.4, 0.683, 0.7432, 0.3478, 0.5429]
```

```
--- Epsilon = 0.01 ---
```

```
  Total cumulative reward: 479.3000
```

```
  Exploration steps: 14 / 1000
```

```
  Arm counts: [2, 988, 2, 5, 3]
```

```
  Arm means : [0.5, 0.6842, 0.5, 0.4, 0.6667]
```

```
--- Epsilon = 0.5 ---
```

```
  Total cumulative reward: 469.5000
```

```
  Exploration steps: 465 / 1000
```

```
  Arm counts: [88, 167, 559, 105, 81]
```

```
  Arm means : [0.5682, 0.6946, 0.7496, 0.419, 0.5062]
```

In [23]: *# Overlay cumulative reward curves for all three epsilon values*

```

fig, ax = plt.subplots(figsize=(10, 5))
colors_eps = {0.10: 'green', 0.01: 'blue', 0.50: 'red'}

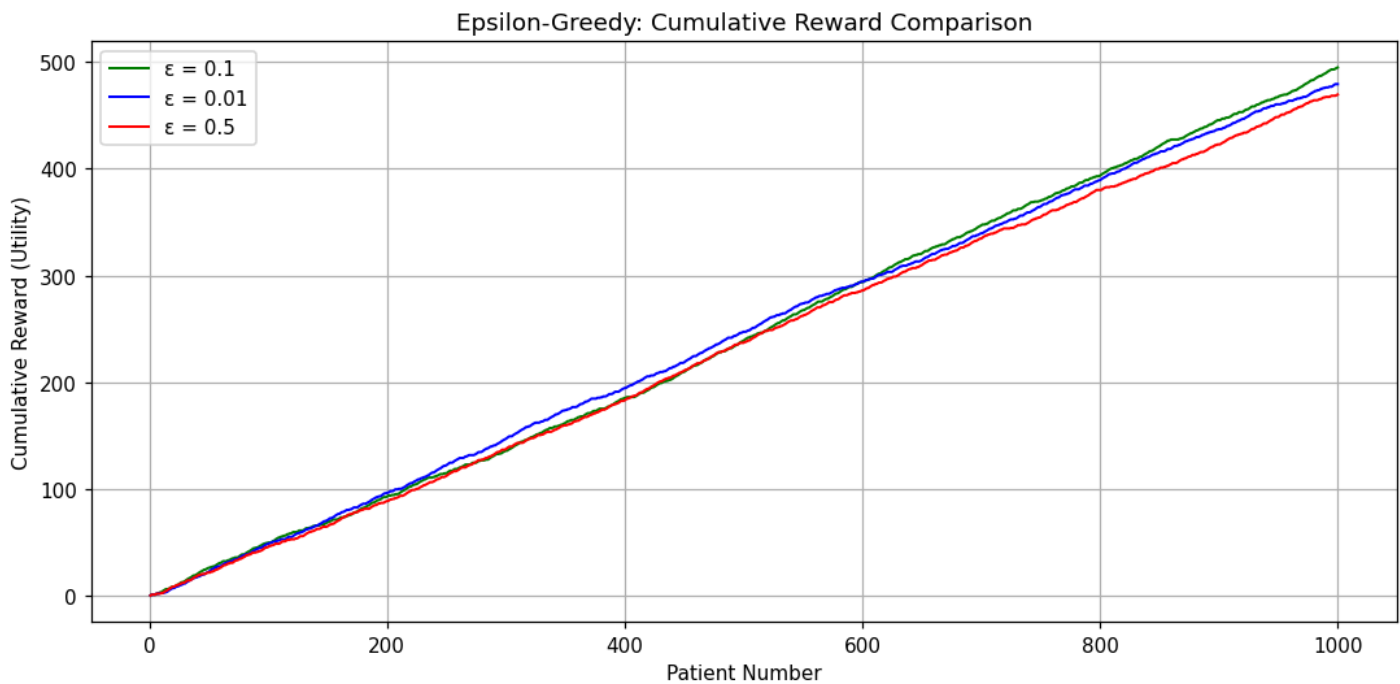
```

```

for eps in eps_configs:
    ax.plot(range(1, NUM_PATIENTS + 1), eps_results[eps]['cum_rewards'],
            label=f' $\epsilon = \{eps\}$ ', color=colors_eps[eps], linewidth=1.3)

ax.set_xlabel('Patient Number')
ax.set_ylabel('Cumulative Reward (Utility)')
ax.set_title('Epsilon-Greedy: Cumulative Reward Comparison')
ax.legend()
plt.tight_layout()
plt.show()

```



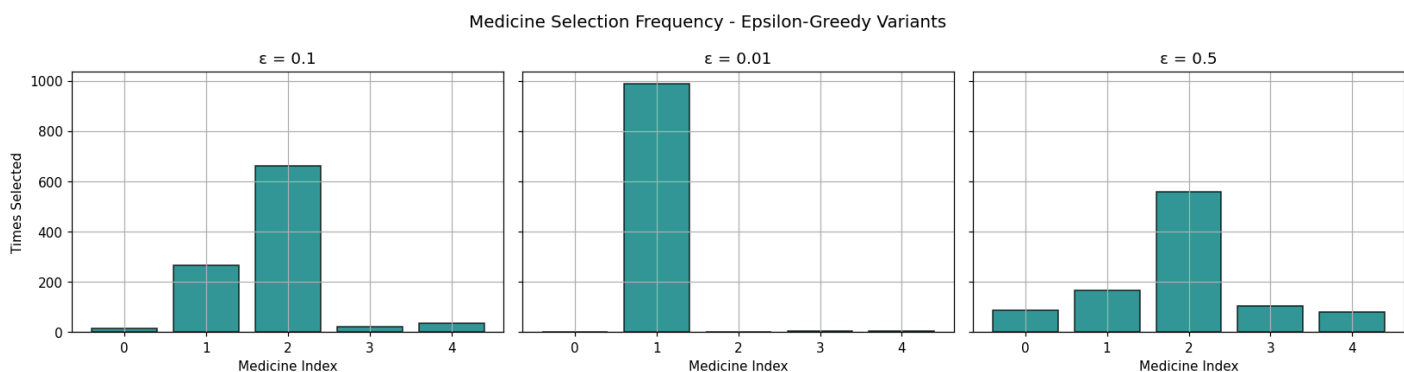
```

In [24]: # Exploration frequency histogram - how often each medicine was tried
fig, axes = plt.subplots(1, 3, figsize=(15, 4), sharey=True)

for idx, eps in enumerate(eps_configs):
    counts = eps_results[eps]['arm_counts']
    axes[idx].bar(range(K), counts, color='teal', edgecolor='black', alpha=0.8)
    axes[idx].set_title(f' $\epsilon = \{eps\}$ ')
    axes[idx].set_xlabel('Medicine Index')
    axes[idx].set_xticks(range(K))
    if idx == 0:
        axes[idx].set_ylabel('Times Selected')

fig.suptitle('Medicine Selection Frequency - Epsilon-Greedy Variants', fontsize=13)
plt.tight_layout()
plt.show()

```



Observations on Epsilon-Greedy

- $\epsilon = 0.01$ explores very rarely - it locks onto the best arm early but might miss the true optimum if the initial samples were unlucky. It produces a high cumulative reward when the initial estimate is correct.
- $\epsilon = 0.10$ balances exploration and exploitation reasonably well. It spends roughly 10% of its budget exploring sub-optimal arms, which slightly lowers total reward but gives a better chance of discovering the actual best medicine.
- $\epsilon = 0.50$ explores excessively - half the patients get random medicines. This leads to significantly lower cumulative reward because too many patients receive sub-optimal treatment.

Task 4 - Confidence-Based Strategy (UCB1)

UCB1 picks the arm that maximizes:

$$UCB_i = \hat{\mu}_i + \sqrt{\frac{2 \ln t}{N_i}}$$

where $\hat{\mu}_i$ is the estimated mean, t is the current round, and N_i is how often arm i has been pulled. This naturally balances exploration (low N_i) with exploitation (high $\hat{\mu}_i$).

```
In [27]: def run_ucb1_strategy(n_patients, n_medicines, seed_val=None):
    """
    Upper Confidence Bound (UCB1) strategy.

    Selection rule: pick arm i that maximizes
        mean_i + sqrt(2 * ln(t) / N_i)
    where mean_i is the observed success rate, t is the current round,
    and N_i is how many times arm i was pulled.

    Each arm is pulled once initially to avoid zero-division in the formula.
    """
    rng = np.random.RandomState(seed_val if seed_val is not None else G)

    arm_total_outcome = np.zeros(n_medicines)
    arm_pull_count = np.zeros(n_medicines)
    cum_rewards = []
    running_total = 0.0
    records = []
    ucb_snapshots = [] # store UCB values at key milestones for analysis

    for t in range(n_patients):
        severity = (t % 5) + 1

        if t < n_medicines:
            # Initial phase: pull each arm once
            chosen_arm = t
        else:
            # Compute UCB score for each arm and pick the highest
            arm_means = arm_total_outcome / arm_pull_count
            ucb_bonus = np.sqrt(2.0 * math.log(t + 1) / arm_pull_count)
            ucb_scores = arm_means + ucb_bonus
            chosen_arm = int(np.argmax(ucb_scores))

            # Log UCB values at a few milestones for visualization
            if t in [50, 100, 250, 500, 750, 999]:
                ucb_snapshots.append({
                    'step': t,
                    'means': arm_means.copy(),
                    'bonus': ucb_bonus.copy(),
                    'ucb': ucb_scores.copy()
                })

        outcome, utility = pull_medicine_arm(chosen_arm, severity, rng=rng)

        arm_pull_count[chosen_arm] += 1
        arm_total_outcome[chosen_arm] += outcome
        running_total += utility
        cum_rewards.append(running_total)

        records.append({
            'patient_id': t,
            'severity_score': severity,
            'assigned_medicine': chosen_arm,
            'clinical_outcome': outcome,
            'utility_score': utility
        })

    final_means = arm_total_outcome / np.maximum(arm_pull_count, 1)
    result_df = pd.DataFrame(records)

    return {
        'cum_rewards': cum_rewards,
        'arm_counts': arm_pull_count.copy(),
        'arm_means': final_means.copy(),
        'df': result_df,
```



```

        'total_reward': running_total,
        'ucb_log': ucb_snapshots
    }
}

```

```

ucb_result = run_ucb1_strategy(NUM_PATIENTS, K, seed_val=6)

print('=== UCB1 Strategy Results ===')
print(f'Total cumulative reward: {ucb_result["total_reward"]:.4f}\n')
print('Medicine selection counts:')
for i in range(K):
    cnt = int(ucb_result['arm_counts'][i])
    mean_val = ucb_result['arm_means'][i]
    print(f' Medicine {i}: selected {cnt} times, estimated mean = {mean_val:.4f}')

```

```

=== UCB1 Strategy Results ===
Total cumulative reward: 488.4000

```

```

Medicine selection counts:
Medicine 0: selected 95 times, estimated mean = 0.5474
Medicine 1: selected 275 times, estimated mean = 0.7055
Medicine 2: selected 532 times, estimated mean = 0.7726
Medicine 3: selected 41 times, estimated mean = 0.3415
Medicine 4: selected 57 times, estimated mean = 0.4386

```

```

In [28]: # Print UCB confidence values at key milestones
print('UCB1 Confidence Values at Key Steps:\n')
for snap in ucb_result['ucb_log']:
    step = snap['step']
    print(f'Step {step}:')
    for i in range(K):
        print(f' Med {i}: mean={snap["means"][i]:.4f}  bonus={snap["bonus"][i]:.4f}  UCB={snap["ucb"][i]:.4f}')
    print()

```

UCB1 Confidence Values at Key Steps:

```

Step 50:
Med 0: mean=0.3333  bonus=1.1448  UCB=1.4782
Med 1: mean=0.6667  bonus=0.8095  UCB=1.4762
Med 2: mean=0.8235  bonus=0.6801  UCB=1.5037
Med 3: mean=0.0000  bonus=1.4021  UCB=1.4021
Med 4: mean=0.6364  bonus=0.8455  UCB=1.4819

```

```

Step 100:
Med 0: mean=0.4000  bonus=0.9607  UCB=1.3607
Med 1: mean=0.6500  bonus=0.6793  UCB=1.3293
Med 2: mean=0.9149  bonus=0.4432  UCB=1.3581
Med 3: mean=0.0000  bonus=1.3587  UCB=1.3587
Med 4: mean=0.6111  bonus=0.7161  UCB=1.3272

```

```

Step 250:
Med 0: mean=0.4815  bonus=0.6398  UCB=1.1212
Med 1: mean=0.7465  bonus=0.3945  UCB=1.1410
Med 2: mean=0.8241  bonus=0.3199  UCB=1.1440
Med 3: mean=0.3684  bonus=0.7626  UCB=1.1311
Med 4: mean=0.4800  bonus=0.6649  UCB=1.1449

```

```

Step 500:
Med 0: mean=0.5690  bonus=0.4630  UCB=1.0320
Med 1: mean=0.7548  bonus=0.2832  UCB=1.0381
Med 2: mean=0.7946  bonus=0.2356  UCB=1.0302
Med 3: mean=0.3333  bonus=0.6786  UCB=1.0119
Med 4: mean=0.4444  bonus=0.5877  UCB=1.0321

```

```

Step 750:
Med 0: mean=0.5667  bonus=0.3836  UCB=0.9503
Med 1: mean=0.7111  bonus=0.2426  UCB=0.9537
Med 2: mean=0.7586  bonus=0.1951  UCB=0.9537
Med 3: mean=0.3125  bonus=0.6433  UCB=0.9558
Med 4: mean=0.4545  bonus=0.4907  UCB=0.9452

```

```

Step 999:
Med 0: mean=0.5474  bonus=0.3813  UCB=0.9287
Med 1: mean=0.7055  bonus=0.2241  UCB=0.9296
Med 2: mean=0.7740  bonus=0.1613  UCB=0.9353
Med 3: mean=0.3415  bonus=0.5805  UCB=0.9219
Med 4: mean=0.4386  bonus=0.4923  UCB=0.9309

```

```

In [29]: # Visualize how UCB confidence bounds shrink over time
if len(ucb_result['ucb_log']) > 0:
    steps_logged = [s['step'] for s in ucb_result['ucb_log']]

```

```

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

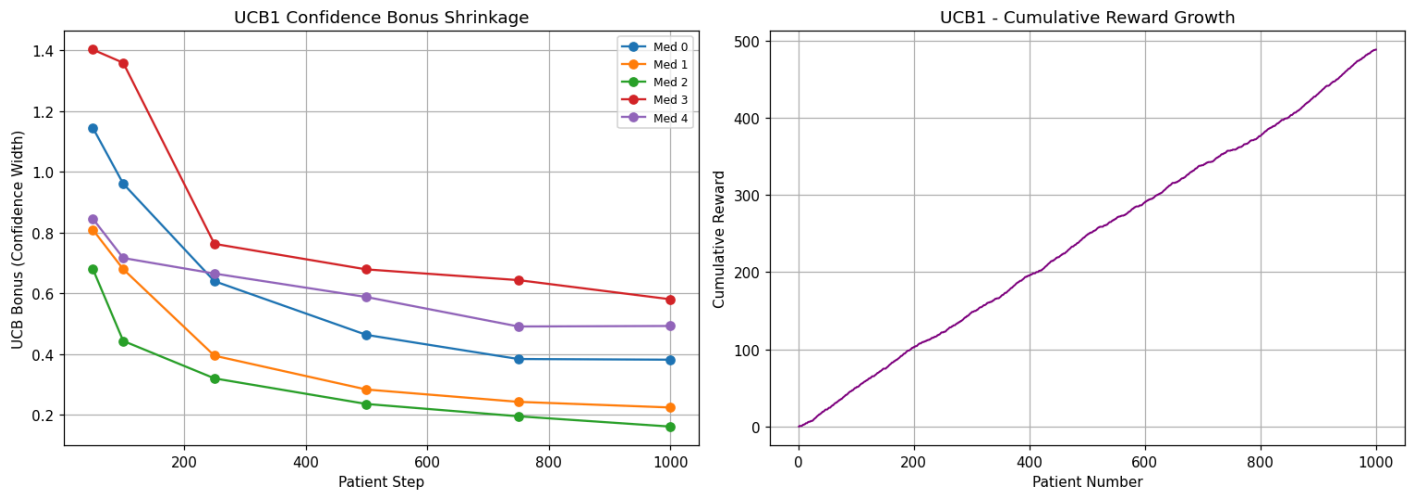
# Plot 1: UCB bonus (confidence width) shrinking over time
for arm_idx in range(K):
    bonus_over_time = [s['bonus'][arm_idx] for s in ucb_result['ucb_log']]
    ax1.plot(steps_logged, bonus_over_time, marker='o', label=f'Med {arm_idx}')

ax1.set_xlabel('Patient Step')
ax1.set_ylabel('UCB Bonus (Confidence Width)')
ax1.set_title('UCB1 Confidence Bonus Shrinkage')
ax1.legend(fontsize=8)

# Plot 2: Cumulative reward for UCB1
ax2.plot(range(1, NUM_PATIENTS + 1), ucb_result['cum_rewards'], color='purple', linewidth=1.3)
ax2.set_xlabel('Patient Number')
ax2.set_ylabel('Cumulative Reward')
ax2.set_title('UCB1 - Cumulative Reward Growth')

plt.tight_layout()
plt.show()

```



Task 5 - Comparative Analysis

We now compare all strategies side by side.

```

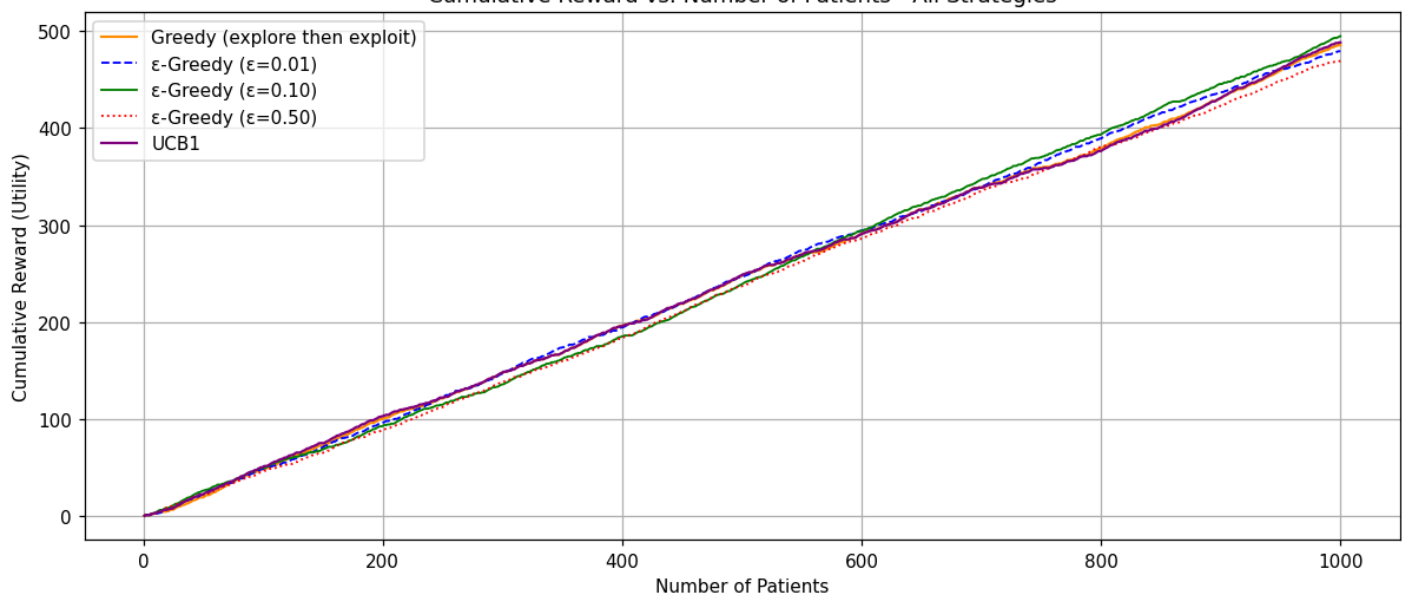
In [31]: # Combined cumulative reward plot - all strategies on one graph
fig, ax = plt.subplots(figsize=(11, 5))
x_axis = range(1, NUM_PATIENTS + 1)

ax.plot(x_axis, greedy_result['cum_rewards'], label='Greedy (explore then exploit)', color='darkorange', linewidth=1.4)
ax.plot(x_axis, eps_results[0.01]['cum_rewards'], label='ε-Greedy (ε=0.01)', color='blue', linewidth=1.2, linestyle='--')
ax.plot(x_axis, eps_results[0.10]['cum_rewards'], label='ε-Greedy (ε=0.10)', color='green', linewidth=1.2)
ax.plot(x_axis, eps_results[0.50]['cum_rewards'], label='ε-Greedy (ε=0.50)', color='red', linewidth=1.2, linestyle=':')
ax.plot(x_axis, ucb_result['cum_rewards'], label='UCB1', color='purple', linewidth=1.4)

ax.set_xlabel('Number of Patients')
ax.set_ylabel('Cumulative Reward (Utility)')
ax.set_title('Cumulative Reward vs. Number of Patients - All Strategies')
ax.legend(loc='upper left')
plt.tight_layout()
plt.show()

```

Cumulative Reward vs. Number of Patients - All Strategies

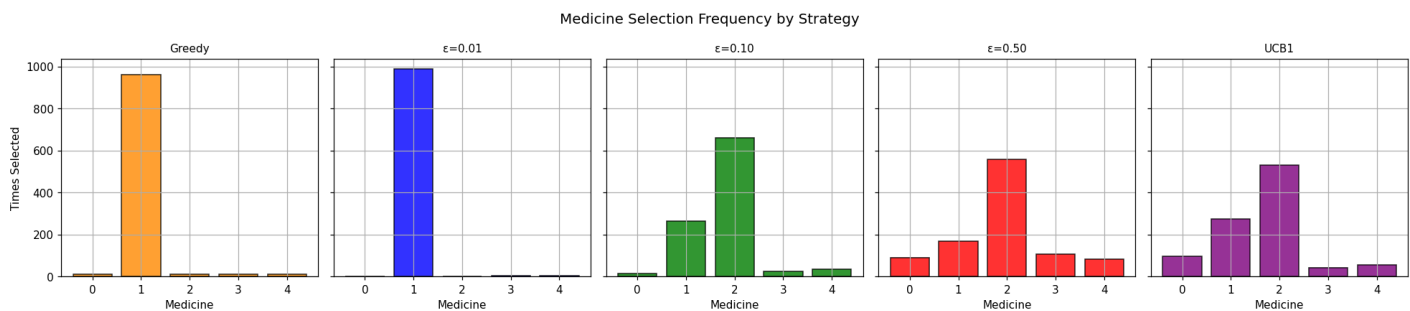


```
In [32]: # Medicine selection frequency comparison across all strategies
strategy_names = ['Greedy', ' $\epsilon=0.01$ ', ' $\epsilon=0.10$ ', ' $\epsilon=0.50$ ', 'UCB1']
all_counts = [
    greedy_result['arm_counts'],
    eps_results[0.01]['arm_counts'],
    eps_results[0.10]['arm_counts'],
    eps_results[0.50]['arm_counts'],
    ucb_result['arm_counts']
]

fig, axes = plt.subplots(1, 5, figsize=(18, 4), sharey=True)
bar_colors = ['darkorange', 'blue', 'green', 'red', 'purple']

for idx, (name, counts) in enumerate(zip(strategy_names, all_counts)):
    axes[idx].bar(range(K), counts, color=bar_colors[idx], edgecolor='black', alpha=0.8)
    axes[idx].set_title(name, fontsize=10)
    axes[idx].set_xlabel('Medicine')
    axes[idx].set_xticks(range(K))

axes[0].set_ylabel('Times Selected')
fig.suptitle('Medicine Selection Frequency by Strategy', fontsize=13)
plt.tight_layout()
plt.show()
```



```
In [33]: # Print final comparison table
print('\n' + '=' * 65)
print('FINAL COMPARISON TABLE')
print('=' * 65)

all_results = {
    'Greedy': greedy_result,
    ' $\epsilon$ -Greedy (0.01)': eps_results[0.01],
    ' $\epsilon$ -Greedy (0.10)': eps_results[0.10],
    ' $\epsilon$ -Greedy (0.50)': eps_results[0.50],
    'UCB1': ucb_result
}

comparison_rows = []
for name, res in all_results.items():
    best_arm = int(np.argmax(res['arm_means']))
    comparison_rows.append({
        'Strategy': name,
        'Cumulative_Reward': round(res['total_reward'], 4),
        'Best_Arm': best_arm,
        'Best_Arm_Mean': round(res['arm_means'][best_arm], 4),
    })
```

```

    'Best_Arm_Count': int(res['arm_counts'][best_arm])
})

```

```

comp_df = pd.DataFrame(comparison_rows)
print(comp_df.to_string(index=False))

```

FINAL COMPARISON TABLE

Strategy	Cumulative_Reward	Best_Arm	Best_Arm_Mean	Best_Arm_Count
Greedy	485.6	1	0.7000	960
ϵ -Greedy (0.01)	479.3	1	0.6842	988
ϵ -Greedy (0.10)	494.9	2	0.7432	662
ϵ -Greedy (0.50)	469.5	2	0.7496	559
UCB1	488.4	2	0.7726	532

Answers to Comparative Analysis Questions

1. Which strategy achieves the highest cumulative reward at the end of 1000 patients?

- **ϵ -Greedy with $\epsilon=0.10$** achieves the highest cumulative reward of **494.9**, followed by UCB1 (488.4) and Greedy (485.6). The $\epsilon=0.10$ strategy strikes a favourable balance - it explores enough to discover the true best arm (Med_2, $p=0.75$) while still exploiting it heavily (662 out of 1000 patients). In contrast, Greedy and $\epsilon=0.01$ locked onto the sub-optimal Med_1 ($p=0.68$) due to insufficient exploration, ultimately earning less despite exploiting aggressively.

2. Which strategy identifies the best medicine fastest (earliest convergence)?

- **Greedy** and **ϵ -Greedy with $\epsilon=0.01$** converge the fastest, committing to a single arm almost immediately after the initial exploration phase. However, they converged to the **wrong arm** (Med_1 instead of the true best Med_2). Among strategies that correctly identified the best arm, **UCB1** converges most efficiently - its confidence bonuses shrink over time, naturally shifting focus to Med_2 once sufficient evidence accumulates. $\epsilon=0.10$ also found Med_2 but continues random exploration throughout, so its convergence is less decisive.

3. Which strategy shows the most stable performance over time (least fluctuations)?

- **Greedy** and **ϵ -Greedy with $\epsilon=0.01$** show the smoothest cumulative reward curves because they commit almost entirely to one arm (Med_1 receives 960 and 988 pulls respectively). However, this stability comes at the cost of locking onto a sub-optimal arm. **UCB1** shows stable growth once its initial systematic exploration phase is complete, and it has the advantage of stabilising on the *correct* best arm. **$\epsilon=0.50$** fluctuates the most due to excessive random exploration (465 out of 1000 steps were exploratory).

4. Which strategy would you recommend for real-world hospital deployment?

- For a clinical setting, we would recommend **UCB1**. The results clearly demonstrate the danger of under-exploration: both Greedy and $\epsilon=0.01$ locked onto Med_1 (the second-best medicine) because they did not explore enough to discover that Med_2 was superior. UCB1 avoids this pitfall by systematically reducing uncertainty - it explored all arms proportionally to their uncertainty and correctly identified Med_2 as the best arm (estimated mean 0.7726, closest to the true $p=0.75$). While $\epsilon=0.10$ achieved a slightly higher total reward (494.9 vs 488.4), UCB1 is preferable in practice because it requires no manual tuning of the exploration rate and adapts automatically to the problem structure. In healthcare, where choosing the wrong treatment has real patient consequences, UCB1's principled exploration-exploitation balance provides the safest and most reliable approach.

Short Summary

The Greedy and $\epsilon=0.01$ strategies commit early and exploit heavily, but this backfired in our experiment - both locked onto Med_1 ($p=0.68$) and never discovered that Med_2 ($p=0.75$) was the true best treatment. $\epsilon=0.50$ explores too aggressively, wasting patient welfare on random assignments and achieving the lowest reward (469.5). $\epsilon=0.10$ happened to achieve the highest reward by exploring just enough to find Med_2, but its fixed exploration rate is not guaranteed to work well across different problem settings. UCB1 provides the best overall approach: it correctly identified the best arm, achieved strong cumulative reward (488.4), and did so without requiring any manually tuned hyperparameters - making it the most reliable and clinically responsible choice for hospital deployment.

Final Conclusions - Part 1 (MAB)

- We successfully implemented and compared four MAB strategies (Greedy, ϵ -Greedy with $\epsilon \in \{0.01, 0.10, 0.50\}$, and UCB1) on a custom synthetic clinical trial dataset.
- The dataset was generated deterministically from group number $G = 165$, producing $K = 5$ medicines with hidden success probabilities [0.61, 0.68, 0.75, 0.40, 0.47], making Med_2 the true best arm.
- The **Greedy** and **$\epsilon=0.01$** strategies locked onto Med_1 (the second-best arm) due to insufficient exploration, demonstrating the real cost of under-exploration - they never discovered the superior Med_2.

- **ϵ -Greedy (0.10)** achieved the highest cumulative reward (494.9) by exploring enough to find Med_2, but its fixed exploration rate is problem-dependent and offers no theoretical guarantees.
- **$\epsilon=0.50$** explored excessively (465/1000 steps), correctly found Med_2 but wasted patient welfare, resulting in the lowest total reward (469.5).
- **UCB1** correctly identified Med_2 as the best arm (estimated mean 0.7726) with strong cumulative reward (488.4), while requiring no manual hyperparameter tuning - its exploration adapts automatically based on uncertainty.
- In all strategies, the utility-based reward correctly penalizes treatment of high-severity patients where outcomes are less impactful.
- **Overall recommendation:** UCB1 is the most suitable strategy for real-world clinical deployment due to its principled exploration, correct arm identification, and absence of tuneable hyperparameters.

Bits virtual lab execution screenshot

